

Abhishek Raghav**Btech CSE****CU24250156**

Experiment No. 1 : Implementation of Fundamental Image Processing Operations using Python and OpenCV

Aim

To develop a comprehensive understanding of fundamental image processing techniques by implementing image acquisition, storage, color space conversion, geometric transformations, and image enhancement operations using Python and OpenCV, thereby establishing the foundation for advanced computer vision applications.

Purpose

Image processing forms the core of modern computer vision systems used in domains such as autonomous driving, healthcare, surveillance, robotics, and industrial automation. Before performing advanced tasks like feature extraction, object detection, or image classification, it is essential to understand how digital images are represented, manipulated, and transformed.

In this experiment, students will implement fundamental image processing operations using Python and OpenCV. They will learn to read, write, convert, resize, and manipulate digital images while understanding how these operations influence image quality and prepare images for subsequent computer vision tasks.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# 1. Load image
image_path = "/Screenshot 2026-08-10 000428.png" # Replace with your saved image filename
img_bgr = cv2.imread(image_path)

if img_bgr is None:
    raise FileNotFoundError("Image file not found. Ensure the image is in the working directory.")

# Convert BGR (OpenCV) to RGB (Matplotlib)
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# 2. Extract and Print Image Properties
height, width, channels = img_bgr.shape
print("=== Image Properties ===")
print(f"Dimensions (H x W) : {height} x {width}")
print(f"Number of Channels : {channels}")
print(f>Data Type : {img_bgr.dtype}")
print(f"Total Pixels/Size : {img_bgr.size} elements ({height * width} pixels)")

# 3. Save Image in Different Formats
cv2.imwrite("output_fare.jpg", img_bgr, [cv2.IMWRITE_JPEG_QUALITY, 95])
cv2.imwrite("output_fare.png", img_bgr, [cv2.IMWRITE_PNG_COMPRESSION, 3])

# 4. Color Space Conversions
img_gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
img_lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)

# 5. Geometric Transformations
# Resize to 256x256
img_resized = cv2.resize(img_rgb, (256, 256), interpolation=cv2.INTER_LINEAR)

# Rotate 90 degrees clockwise
center = (width // 2, height // 2)
rot_matrix = cv2.getRotationMatrix2D(center, angle=-90, scale=1.0)
img_rotated = cv2.warpAffine(img_rgb, rot_matrix, (width, height))

# Horizontal and Vertical Flips
img_flip_h = cv2.flip(img_rgb, 1)
img_flip_v = cv2.flip(img_rgb, 0)

# 6. Image Complement (Negative)
img_negative = 255 - img_rgb
```

```
# 7. Crop Region of Interest (ROI) - Isolating the 'Total amount paid' section
# Cropping the bottom 30% of the image
roi_ymin, roi_ymax = int(height * 0.70), height
roi_xmin, roi_xmax = 0, width
img_roi = img_rgb[roi_ymin:roi_ymax, roi_xmin:roi_xmax]

# 8. Visualization using Matplotlib Grid
fig, axes = plt.subplots(3, 3, figsize=(12, 12))

axes[0, 0].imshow(img_rgb)
axes[0, 0].set_title("1. Original Image (RGB)")

axes[0, 1].imshow(img_gray, cmap="gray")
axes[0, 1].set_title("2. Grayscale Conversion")

axes[0, 2].imshow(img_hsv)
axes[0, 2].set_title("3. HSV Color Space")

axes[1, 0].imshow(img_lab)
axes[1, 0].set_title("4. LAB Color Space")

axes[1, 1].imshow(img_resized)
axes[1, 1].set_title("5. Resized (256x256)")

axes[1, 2].imshow(img_rotated)
axes[1, 2].set_title("6. Rotated (-90°)")

axes[2, 0].imshow(img_flip_h)
axes[2, 0].set_title("7. Horizontal Flip")

axes[2, 1].imshow(img_negative)
axes[2, 1].set_title("8. Negative (Inverted)")

axes[2, 2].imshow(img_roi)
axes[2, 2].set_title("9. Cropped ROI (Total Fare)")

for ax in axes.ravel():
    ax.axis("off")

plt.tight_layout()
plt.show()
```

```

=== Image Properties ===
Dimensions (H x W) : 325 x 323
Number of Channels : 3
Data Type : uint8
Total Pixels/Size : 314925 elements (104975 pixels)

```

1. Original Image (RGB)

Fare and payment details

Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

2. Grayscale Conversion

Fare and payment details

Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

3. HSV Color Space

Fare and payment details

Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

4. LAB Color Space

Fare and payment details

Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

5. Resized (256x256)

Fare and payment details

Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

6. Rotated (-90°)

Fare and payment details	
Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

7. Horizontal Flip

Fare and payment details

00.0015	▾ Ticket fare
18.15	▾ IRCTC charges
00.83	▲ Additional charges
00.00	to (incl. of GST) Agent service charge
815 24 ₹18	Free cancellation charge (incl. of GST) T&C
206.15	Total amount paid
212.15	

8. Negative (Inverted)

Fare and payment details

Ticket fare ▾	₹150.00
IRCTC charges ▾	₹18.15
Additional charges ▲	₹38.00
Agent service charge (incl. of GST)	₹20.00
Free cancellation charge (incl. of GST)	T&C ₹24 ₹18
Total amount paid	₹206.15
	₹212.15

9. Cropped ROI (Total Fare)

Free cancellation charge (incl. of GST) T&C ₹24 ₹18

Total amount paid **₹206.15**
₹212.15

Answers to Experiment Questions

1. What is a digital image? Differentiate between grayscale and color images.

A digital image is a 2D matrix of discrete numerical elements called pixels, where each pixel value represents an intensity or color level at a specific spatial coordinate (x, y) .

Grayscale vs. Color:

Grayscale Images: Represented by a single 2D channel (matrix), where pixel intensities range from 0 (pure black) to 255 (pure white) in standard 8-bit depth.

Color Images: Represented by a multi-channel 3D array (typically 3 channels: Red, Green, Blue), where a combination of the three channel intensities creates the full color palette.

2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

RGB (Red, Green, Blue): An additive color model standard in computer graphics, monitors, and web displays.

BGR (Blue, Green, Red): Identical to RGB in structure, but with inverted channel ordering; default format used internally by OpenCV.

HSV (Hue, Saturation, Value): Separates chromatic content from lighting. Hue defines the color angle ($0^\circ - 360^\circ$), Saturation represents color vibrancy/purity, and Value represents lightness/brightness. Widely used for color-based thresholding and object tracking.

LAB ($L^*a^*b^*$): A perceptually uniform color space designed to mimic human vision. L represents Lightness, a defines the green-to-magenta spectrum, and b defines the blue-to-yellow spectrum. Separates luminance completely from color chrominance.

3. What is the purpose of converting an image to grayscale before further processing?

- **Dimensionality Reduction:** Reduces data volume by a factor of 3 (from 3 channels to 1), significantly reducing memory usage and computational latency.
- **Algorithm Compatibility:** Fundamental algorithms such as Canny edge detection, Sobel filtering, thresholding, and morphological operations require single-channel intensity inputs to compute spatial gradients.
- **Invariance to Color Shifts:** Eliminates variations caused by illumination colors and white-balance differences, allowing models to focus purely on shapes, edges, and structural texture.

4. Explain the concept of image complement (negative image) and mention its practical applications.

Concept: An image negative is produced by subtracting each pixel value from the maximum allowable intensity (

$I_{\text{neg}}(x, y) = 255 - I(x, y)$ for 8-bit images). Dark regions become bright and bright regions become dark.

Practical Applications:

- **Medical Imaging:** Enhancing the visibility of white bone structures, calcifications, and micro-fractures in X-rays or CT scans against dark backgrounds.
- **Microscopy & Forensics:** Revealing fine structural details, latent fingerprints, or biological specimen boundaries in high-contrast or inverted illumination.

5. Differentiate between image resizing, cropping, and scaling.

Resizing: Changes the total pixel dimensions (width $\times$ height) of the whole image, which may alter the aspect ratio unless proportional scaling is applied.

Cropping: Selects and extracts a sub-region (Region of Interest) of the image while discarding all outer pixel data.

Scaling: A specific type of resizing that expands (upsampling) or shrinks (downsampling) an image by a uniform scaling factor (s_x, s_y), typically preserving the original aspect ratio.

6. What is a Region of Interest (ROI), and why is it important in computer vision?

An ROI is a localized sub-area of an image containing the specific target or structure under analysis (e.g., a pedestrian bounding box, a vehicle license plate, or a tumor boundary).

Importance:

- Drastically reduces processing time by focusing heavy computation only on relevant pixel patches.
- Filters out background noise, clutter, and irrelevant features, improving model accuracy and detection precision.

7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

Optimized C/C++ Backend: Core operations are written in highly optimized C/C++ with SIMD/AVX vectorization and multi-threading, executing significantly faster than pure Python alternatives.

Hardware Acceleration: Native support for OpenCL, CUDA (GPU acceleration), and embedded hardware (VPU/NPU).

Comprehensive CV Stack: Contains end-to-end computer vision algorithms out of the box—including camera calibration, optical flow, geometric warping, object tracking, and DNN model inference.

8. Explain how image resolution and pixel intensity influence image quality.

Resolution (Spatial Density): The total number of pixels across the width and height (or DPI/PPI). Higher spatial resolution preserves finer spatial details, sharp edges, and small objects without pixelation artifacts.

Pixel Intensity (Bit Depth / Radiometric Resolution): The dynamic range of values a pixel can take (e.g., 8-bit vs. 16-bit). Higher bit depth provides finer contrast transitions, smoother gradients, and prevents posterization/color banding in shadow and highlight regions.

9. Mention five real-world applications where basic image preprocessing is an essential step.

1. Autonomous Vehicles: Color conversion (e.g., HLS/HSV for lane line extraction), geometric undistortion of camera lenses, and ROI cropping for traffic sign classification.
2. Medical Diagnostics: Grayscale conversion, contrast stretching (CLAHE), and image negative generation for X-ray and MRI scans.
3. Optical Character Recognition (OCR) / Document Scanning: Binarization, grayscale conversion, deskewing (rotation), and noise removal to make text readable.
4. Biometric Face Recognition & Verification: Face ROI detection, affine alignment, resizing to standard tensor dimensions, and